

Portfolioteil 2 – Erarbeitungs-/Reflexionsphase

Kurze Zusammenfassung der Implementierung
DLBDSPBDM01_D – Data-Mart-Erstellung in SQL

Tizian Senger

Matrikelnummer: IU14143428

<https://github.com/TizianSenger/IUDatabaseProject>

26. September 2026

1. Verwendetes Datenbankmanagementsystem

Für die Umsetzung wurde **SQLite 3** gewählt. Die Aufgabenstellung erlaubt ausdrücklich jedes Datenbankmanagementsystem, das SQL als Basissprache verwendet – SQLite erfüllt das vollständig (Standard-DDL/DML, JOINS, FOREIGN KEY, CHECK, TRIGGER). Der Ausschlag für SQLite war die serverlose Architektur: Die gesamte Datenbank liegt als einzelne Datei vor, benötigt keine Installation eines Datenbankservers und lässt sich direkt über eine VS-Code-Erweiterung einsehen, was Nachvollziehbarkeit und Weitergabe (z. B. im GitHub-Repository) erheblich vereinfacht. Da SQLite keine nativen ENUM-Typen kennt, wurden alle ehemals als ENUM geplanten Felder (Rolle, Zustand, Status, Versandart) konsequent über CHECK-Constraints mit erlaubter Werteliste abgebildet – eine direkte Umsetzung des Tutor-Feedbacks zu konsequenten Constraints.

2. Umsetzung des ER-Modells

Die 13 Entitäten aus dem überarbeiteten ER-Modell (Phase 1) wurden 1:1 als Tabellen umgesetzt. Die binären M:N-Beziehungen *Verfasst* (Buch–Autor) und *Gehört zu* (Buch–Genre) wurden als eigene Verknüpfungstabellen `buch_autor` und `buch_genre` mit zusammengesetztem Primärschlüssel aufgelöst. Die zwei Dreifachbeziehungen `LEIHT_AUS` und `VERFUEGBAR_AN` wurden als eigene Tabellen (`ausleihe`, `verfuegbarkeit`) mit Fremdschlüsseln auf alle drei beteiligten Tabellen realisiert. Die Entität `BEWERTUNG` trägt statt einer eigenen Dreifachbeziehung einen eindeutigen (UNIQUE) Fremdschlüssel `ausleihe_id` und ist damit strukturell an genau einen abgeschlossenen Ausleihvorgang gekoppelt (Tutor-Feedback Punkt 1). Zwei Trigger – einer verhindert die Ausleihe eines eigenen Exemplars, der andere erlaubt eine Bewertung erst nach erfolgter Rückgabe (Details siehe .sql-Datei) – setzen Geschäftsregeln durch, die SQLite-CHECK-Constraints allein nicht abbilden können, da diese keine Subqueries auf andere Tabellen erlauben.

3. Normalisierung

Das Schema liegt in der 3. Normalform vor: Jede Tabelle hat einen Primärschlüssel, alle Nicht-Schlüsselattribute hängen voll funktional von diesem ab (2NF), und es bestehen keine transitiven Abhängigkeiten (3NF) – Autor, Verlag, Genre und Sprache sind konsequent in eigene Tabellen ausgelagert statt als Text in `buch` wiederholt zu werden. Die Trennung von `buch` (Werk) und `buchexemplar` (physische Kopie mit Eigentümer:in und Zustand) vermeidet zusätzlich, dass Werkdaten pro Exemplar redundant gespeichert werden müssen, wenn mehrere Nutzende dasselbe Buch besitzen.

4. Dummy-Daten und Testfälle

Alle 17 Tabellen enthalten mindestens 10 Datensätze (insgesamt über 190 Zeilen), thematisch konsistent befüllt (deutschsprachige Buchklassiker, deutsche Städte/Adressen, plausible Ausleih- und Bewertungshistorie). Die Befüllung sowie sämtliche Testfälle wurden über ein Python-`sqlite3`-Skript

automatisiert gegen die `.sql`-Datei ausgeführt und verifiziert. Für jede Entität liegt mindestens eine erfolgreiche Testabfrage vor (siehe Präsentation). Zusätzlich wurden gezielt **sechs ungültige Fälle** formuliert und tatsächlich ausgeführt: eine Sternebewertung außerhalb 1–5, negative Versandkosten, ein Zeitslot mit `startzeit >= endzeit`, eine Ausleihe mit nicht existierender `nutzer_id`, eine Selbstausleihe sowie eine Bewertung zu einer noch nicht zurückgegebenen Ausleihe. Alle sechs wurden von der Datenbank korrekt mit der erwarteten Fehlermeldung abgelehnt (Tutor-Feedback Punkt 4).

5. Herausforderungen und Erkenntnisse

Die größte konzeptionelle Herausforderung war, das Tutor-Feedback aus Phase 1 nicht nur in die Dokumentation, sondern strukturell in das Schema zu übersetzen: Ohne die Trennung von Buch und Buchexemplar hätte kein zweites Exemplar desselben Werks bei einer anderen Person abgebildet werden können; ohne die eindeutige Kopplung von Bewertung an `ausleihe_id` wäre eine Bewertung ohne echten Ausleihvorgang möglich gewesen. SQLite erzwang zudem, Geschäftsregeln explizit zu machen: Da CHECK-Constraints keine Subqueries erlauben, mussten crossreferenzierende Regeln (kein Selbstausleih, keine verfrühte Bewertung) als Trigger formuliert werden – was die Regeln am Ende expliziter und leichter nachvollziehbar macht als eine rein anwendungsseitige Prüfung. Für Phase 3 ist geplant, zusätzliche Indizes für die häufig gefilterten Spalten (`ausleihe.status`, `buchexemplar.buch_id`) zu ergänzen und die Umkreissuche über eine Haversine-Berechnung auf Basis der `standort`-Koordinaten zu konkretisieren.